PROG6212

PROGRAMMING 2B

PoE PART 3 / FINAL POE

RACEDAY MVC WEB APPLICATION, AZURE BLOB STORAGE & DOCKER

EXTREMELY DETAILED BEGINNER-FRIENDLY STEP-BY-STEP IMPLEMENTATION GUIDE

Official Part 3 assessment value: 100 marks | Final PoE total: 300 marks

How to use this guide

Work from the beginning to the end. Do not start Docker until the MVC application works locally and successfully communicates with the Part 2 API. Every code sample is a reference implementation: adapt endpoint names and DTO properties to match the Part 1 plan and Part 2 API you actually built.

The most important Part 3 rule

The MVC project is the web front end. It must consume the Part 2 API for data operations. The MVC project must not bypass the API and connect directly to SQL Server. Azure file handling must also go through the Part 2 API.

Prepared from the official 2026 PROG6212 PoE Part 3 brief and rubric, the existing PROG6212 Part 2 student guide, and current official Docker/Microsoft documentation.

Guide map

Stage	What you will complete
1	Understand the official Part 3 requirements and rubric
2	Check the Part 2 API before adding the MVC project
3	Create RaceDay.Mvc and shared references
4	Configure API communication, session state and the API session cookie
5	Build register, login, logout and role-aware navigation
6	Build the Organiser dashboard and management pages
7	Build the Participant pages and enrolment/results flow
8	Add event banner and profile-image uploads through Azure Blob Storage
9	Beginner Docker bootcamp: install, verify, images, containers, ports and commands
10	Dockerise RaceDay.Mvc and connect the container to the host API
11	Troubleshoot Docker and container-to-API problems
12	Update GitHub Actions, README, commits and video evidence
13	Rubric-by-rubric final testing checklist

Official Part 3 summary

The official brief requires an ASP.NET Core MVC application consuming the Part 2 RESTful API, session-based authentication and role-specific interfaces, Organiser and Participant functionality, Azure Blob Storage through the API, Docker containerisation of the MVC application, consistent UI design, GitHub/CI/CD and a video presentation.

1. What Part 3 actually requires

Start here so you know what is compulsory and what is only a recommended implementation choice.

Part 3 is the final front-end and deployment stage of RaceDay. Part 1 planned the system. Part 2 built the API. Part 3 builds the MVC website that people actually use and then packages that MVC website inside Docker.

Official criterion	Marks	What top-band work looks like
Correct Submission	5	Clear folder structure, complete README, Docker run command, CI screenshot, video link
GitHub and CI/CD	10	20+ meaningful commits and successful workflow including the MVC project
Authentication and Role Navigation	10	Register/login work, session stores user and role, distinct menus, logout clears session, protected pages redirect
Organiser Interface	15	Dashboard, Events CRUD, Categories, Enrolments, Results all work
Participant Interface	15	Upcoming events, event details/enrolment, My Events, My Results and Profile all work
Azure Blob Storage	10	MVC sends file to API, API uploads to Azure, URL saved, image displayed
Docker	10	MVC builds and runs cleanly in container; README contains working one-command run instruction
UI Design	10	Professional RaceDay sporting theme, responsive layout, status colours and consistent navigation
Video Presentation	15	Full working demo with code explanation, Blob integration and Docker setup

Not optional

The Docker requirement is the MVC application running inside a container. Docker is not a substitute for building the MVC pages. The containerised application must behave like the local application.

Recommended implementation in this guide: ASP.NET Core MVC on .NET 8, RaceDay.Contracts reused by both API and MVC, HttpClient/IHttpClientFactory for API calls, ASP.NET Core session in the MVC app, Bootstrap for UI, Azure.Storage.Blobs inside the API, and Docker Desktop with Linux containers.

2. Before you create Part 3: prove Part 2 still works

Do not create MVC pages against a broken API. Your Part 2 API is now the back end for the whole final system. Before touching Part 3, run through the following check.

- RaceDay.Api builds with no errors.
- SQL Server connection works.
- Swagger opens.
- Registration works.
- Login works and creates the API session.
- GET /api/events works.
- Organiser can create/update/delete an Event.
- Participant can enrol.
- Organiser can view enrolments.
- Organiser can capture results.
- Participant can view own results.
- All Part 2 unit tests pass.

2.1 Keep the existing solution; add Part 3 to it

```
RaceDay
|-- docs
|-- RaceDay.Api
|-- RaceDay.Contracts
|-- RaceDay.Api.Tests
|-- RaceDay.Mvc          <-- Part 3 web front end
|-- .github/workflows
|-- README.md
`-- RaceDay.sln
```

Do not duplicate the business logic

The MVC application should ask the API to perform operations. If you copy EF Core DbContext/database code into RaceDay.Mvc, you are bypassing the required architecture.

3. Create the RaceDay MVC project step by step

1. Open the existing RaceDay solution in Visual Studio 2022.
2. In Solution Explorer, right-click the solution name, not one of the projects.
3. Choose Add > New Project.
4. Search for ASP.NET Core Web App (Model-View-Controller).
5. Select C# and click Next.
6. Name the project RaceDay.Mvc.
7. Choose the same .NET version used by your Part 2 solution (the class guide uses .NET 8).
8. For Authentication Type choose None because RaceDay authentication is handled through your Part 2 API and your own session logic.
9. Create the project.
10. Right-click RaceDay.Mvc > Add > Project Reference and tick RaceDay.Contracts.

3.1 What MVC means

Part	Simple meaning	RaceDay example
Model	Data used by the page	Login form, EventDto, DashboardVm
View	The .cshtml page the user sees	Login.cshtml, Index.cshtml, MyResults.cshtml
Controller	Receives browser request, calls API, chooses a view	AuthController, EventsController, OrganiserController

3.2 Recommended folders

```
RaceDay.Mvc
|-- Controllers
|   |-- AuthController.cs
|   |-- EventsController.cs
|   |-- OrganiserController.cs
|   `-- ParticipantController.cs
|-- Services
|   `-- RaceDayApiClient.cs
|-- ViewModels
|   |-- LoginVm.cs
|   |-- RegisterVm.cs
|   |-- OrganiserDashboardVm.cs
|   `-- EventFormVm.cs
|-- Views
|   |-- Auth
|   |-- Events
|   |-- Organiser
|   |-- Participant
|   `-- Shared
|-- wwwroot
|-- appsettings.json
|-- Program.cs
`-- Dockerfile
```

4. Configure the MVC application to call the Part 2 API

4.1 Add the API URL to appsettings.json

```
{
 "ApiSettings": {
  "BaseUrl": "http://localhost:5000/"
 },
 "Logging": {
  "LogLevel": {
   "Default": "Information",
   "Microsoft.AspNetCore": "Warning"
  }
 },
 "AllowedHosts": "*"
}
```

Why configuration instead of hard-coding?

The local MVC app and Docker container use different addresses to reach the same API. If the URL is kept in configuration, Docker can override it without changing the C# code.

4.2 Configure Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

// Session stores the MVC user's ID, name and role between page requests.
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});

// Allows a service class to read the current MVC session.
builder.Services.AddHttpContextAccessor();

// Register one named HttpClient for the Part 2 API.
builder.Services.AddHttpClient("RaceDayApi", client =>
{
    string baseUrl = builder.Configuration["ApiSettings:BaseUrl"]
        ?? throw new InvalidOperationException("ApiSettings:BaseUrl is missing.");

    client.BaseAddress = new Uri(baseUrl);
});

builder.Services.AddScoped<RaceDayApiClient>();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
```

```
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();

// IMPORTANT: session must be enabled before controller endpoints need it.
app.UseSession();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Middleware order

ASP.NET Core session must be configured with AddSession and activated with UseSession. In a normal MVC pipeline, call UseSession after routing and before mapped controller endpoints need the session.

4.3 Why the API session cookie matters

The Part 2 guide used server-side API session state. When the MVC app logs in through the API, the API sends back its own session cookie. If the MVC server ignores that cookie, the next protected API call may return 401 because the API no longer knows which user is making the request.

The following beginner-friendly client stores the API session cookie inside the MVC user session and sends it back on later API requests. That keeps the two sessions separate: MVC session controls the website experience; the API session proves identity to the protected API endpoints.

4.4 Create Services/RaceDayApiClient.cs

```
using System.Net.Http.Json;
using System.Text.Json;

namespace RaceDay.Mvc.Services;

public class RaceDayApiClient
{
    private readonly IHttpClientFactory _factory;
    private readonly IHttpContextAccessor _httpContextAccessor;

    private static readonly JsonSerializerOptions JsonOptions = new()
    {
        PropertyNameCaseInsensitive = true
    };

    public RaceDayApiClient(IHttpClientFactory factory, IHttpContextAccessor accessor)
    {
```

```
        _factory = factory;
        _httpContextAccessor = accessor;
    }

    private HttpClient CreateClient()
    {
        return _factory.CreateClient("RaceDayApi");
    }

    private HttpRequestMessage CreateRequest(HttpMethod method, string url)
    {
        var request = new HttpRequestMessage(method, url);

        string? apiCookie = _httpContextAccessor.HttpContext?
            .Session.GetString("ApiSessionCookie");

        if (!string.IsNullOrWhiteSpace(apiCookie))
        {
            request.Headers.TryAddWithoutValidation("Cookie", apiCookie);
        }

        return request;
    }

    public async Task<HttpResponseMessage> SendAsync(
        HttpMethod method, string url, object? body = null)
    {
        var client = CreateClient();
        using var request = CreateRequest(method, url);

        if (body != null)
            request.Content = JsonContent.Create(body);

        return await client.SendAsync(request);
    }

    public async Task<T?> ReadAsync<T>(HttpResponseMessage response)
    {
        if (!response.IsSuccessStatusCode) return default;
        string json = await response.Content.ReadAsStringAsync();
        return JsonSerializer.Deserialize<T>(json, JsonOptions);
    }

    public static string? ExtractSessionCookie(HttpResponseMessage response)
    {
        if (!response.Headers.TryGetValues("Set-Cookie", out var values))
            return null;

        string? sessionHeader = values.FirstOrDefault(v =>
            v.StartsWith(".AspNetCore.Session=", StringComparison.OrdinalIgnoreCase));

        // A request Cookie header needs only name=value, not Path/HttpOnly/etc.
        return sessionHeader?.Split(';', 2)[0];
    }
}
```

5. Registration, login, logout and role-aware navigation

5.1 View models

```
using System.ComponentModel.DataAnnotations;

public class LoginVm
{
    [Required, EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required, DataType(DataType.Password)]
    public string Password { get; set; } = string.Empty;
}

public class RegisterVm
{
    [Required] public string FirstName { get; set; } = string.Empty;
    [Required] public string LastName { get; set; } = string.Empty;
    [Required, EmailAddress] public string Email { get; set; } = string.Empty;
    [Required, DataType(DataType.Password)] public string Password { get; set; } = string.Empty;
    [Required] public string Role { get; set; } = "Participant";
}
```

5.2 AuthController reference implementation

```
using Microsoft.AspNetCore.Mvc;
using RaceDay.Contracts;
using RaceDay.Mvc.Services;

public class AuthController : Controller
{
    private readonly RaceDayApiClient _api;
    public AuthController(RaceDayApiClient api) => _api = api;

    [HttpGet]
    public IActionResult Register() => View();

    [HttpPost]
    public async Task<IActionResult> Register(RegisterVm vm)
    {
        if (!ModelState.IsValid) return View(vm);

        var request = new RegisterRequest
        {
            FirstName = vm.FirstName,
            LastName = vm.LastName,
            Email = vm.Email,
            Password = vm.Password,
            Role = vm.Role
        };

        var response = await _api.SendAsync(HttpMethod.Post, "api/auth/register", request);
        if (!response.IsSuccessStatusCode)
        {
```

```
            ModelState.AddModelError("", await response.Content.ReadAsStringAsync());
            return View(vm);
        }

        TempData["Success"] = "Registration successful. Please log in.";
        return RedirectToAction(nameof(Login));
    }

    [HttpGet]
    public IActionResult Login() => View();

    [HttpPost]
    public async Task<IActionResult> Login(LoginVm vm)
    {
        if (!ModelState.IsValid) return View(vm);

        var response = await _api.SendAsync(HttpMethod.Post, "api/auth/login",
            new LoginRequest { Email = vm.Email, Password = vm.Password });

        if (!response.IsSuccessStatusCode)
        {
            ModelState.AddModelError("", "Invalid email or password.");
            return View(vm);
        }

        var auth = await _api.ReadAsync<AuthResponse>(response);
        if (auth == null)
        {
            ModelState.AddModelError("", "Login response could not be read.");
            return View(vm);
        }

        string? apiCookie = RaceDayApiClient.ExtractSessionCookie(response);
        if (!string.IsNullOrWhiteSpace(apiCookie))
            HttpContext.Session.SetString("ApiSessionCookie", apiCookie);

        HttpContext.Session.SetInt32("UserId", auth.UserId);
        HttpContext.Session.SetString("Role", auth.Role);
        HttpContext.Session.SetString("DisplayName", auth.FirstName ?? auth.Email);

        if (auth.Role.Equals("Organiser", StringComparison.OrdinalIgnoreCase))
            return RedirectToAction("Index", "Organiser");

        return RedirectToAction("Index", "Participant");
    }

    [HttpPost]
    public async Task<IActionResult> Logout()
    {
        await _api.SendAsync(HttpMethod.Post, "api/auth/logout");
        HttpContext.Session.Clear();
        return RedirectToAction("Index", "Home");
    }
}
```

Adapt AuthResponse

Your Part 2 AuthResponse property names may differ. Use the exact properties your API returns. The key values Part 3 needs are the user ID, role and a useful display name.

5.3 Protect a page without adding another authentication framework

```
private bool IsLoggedIn() => HttpContext.Session.GetInt32("UserId").HasValue;

private bool IsOrganiser() =>
    string.Equals(HttpContext.Session.GetString("Role"), "Organiser",
        StringComparison.OrdinalIgnoreCase);

public IActionResult Index()
{
    if (!IsLoggedIn())
        return RedirectToAction("Login", "Auth");

    if (!IsOrganiser())
        return RedirectToAction("AccessDenied", "Home");

    return View();
}
```

5.4 Role-aware navigation in Views/Shared/_Layout.cshtml

```
@{
    var role = Context.Session.GetString("Role");
    var userId = Context.Session.GetInt32("UserId");
}

@if (userId == null)
{
    <a asp-controller="Auth" asp-action="Login">Login</a>
    <a asp-controller="Auth" asp-action="Register">Register</a>
}
else if (role == "Organiser")
{
    <a asp-controller="Organiser" asp-action="Index">Dashboard</a>
    <a asp-controller="Organiser" asp-action="Events">Events</a>
    <a asp-controller="Organiser" asp-action="Enrolments">Enrolments</a>
    <a asp-controller="Organiser" asp-action="Results">Results</a>
}
else
{
    <a asp-controller="Participant" asp-action="Index">Events</a>
    <a asp-controller="Participant" asp-action="MyEvents">My Events</a>
    <a asp-controller="Participant" asp-action="MyResults">My Results</a>
    <a asp-controller="Participant" asp-action="Profile">Profile</a>
}
```

6. Build the Organiser interface

The Organiser top band requires all major pages to be functional: dashboard, event management, categories, enrolments and results entry. Build them one at a time and test the API call after each page.

6.1 Organiser dashboard

The dashboard should summarise events the logged-in organiser owns, show total enrolments per event and highlight upcoming dates. If your API does not yet expose a purpose-built dashboard endpoint, you can compose the dashboard from the organiser event endpoint plus enrolment counts returned by the API.

```
public class OrganiserDashboardVm
{
    public List<OrganiserEventSummaryVm> Events { get; set; } = new();
}

public class OrganiserEventSummaryVm
{
    public int EventId { get; set; }
    public string Name { get; set; } = string.Empty;
    public DateTime EventDate { get; set; }
    public int EnrolmentCount { get; set; }
}
```

6.2 Event management controller pattern

```
public async Task<IActionResult> Events()
{
    if (!IsOrganiser()) return RedirectToAction("Login", "Auth");

    var response = await _api.SendAsync(HttpMethod.Get, "api/events/mine");
    var events = await _api.ReadAsync<List<EventDto>>(response) ?? new();
    return View(events);
}

[HttpGet]
public IActionResult CreateEvent()
{
    if (!IsOrganiser()) return RedirectToAction("Login", "Auth");
    return View();
}

[HttpPost]
public async Task<IActionResult> CreateEvent(CreateEventRequest model)
{
    if (!IsOrganiser()) return RedirectToAction("Login", "Auth");
    if (!ModelState.IsValid) return View(model);

    var response = await _api.SendAsync(HttpMethod.Post, "api/events", model);
    if (!response.IsSuccessStatusCode)
    {
        ModelState.AddModelError("", await response.Content.ReadAsStringAsync());
        return View(model);
    }
```

```
    return RedirectToAction(nameof(Events));
}
```

Create/Edit form fields that must be present

- Name
- Description
- Date
- Location
- Distance
- Event type: Run, Walk or Cycle
- Banner image upload (added in the Blob Storage section)

6.3 Example CreateEvent.cshtml

```
@model RaceDay.Contracts.CreateEventRequest

<h1>Create Event</h1>
<form asp-action="CreateEvent" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <label asp-for="Name"></label>
    <input asp-for="Name" class="form-control" />
    <span asp-validation-for="Name" class="text-danger"></span>

    <label asp-for="Description"></label>
    <textarea asp-for="Description" class="form-control"></textarea>

    <label asp-for="EventDate"></label>
    <input asp-for="EventDate" type="date" class="form-control" />

    <label asp-for="Location"></label>
    <input asp-for="Location" class="form-control" />

    <label asp-for="DistanceKm"></label>
    <input asp-for="DistanceKm" class="form-control" />

    <label asp-for="EventType"></label>
    <select asp-for="EventType" class="form-select">
        <option value="Run">Run</option>
        <option value="Walk">Walk</option>
        <option value="Cycle">Cycle</option>
    </select>

    <button class="btn btn-success mt-3" type="submit">Create Event</button>
</form>
```

6.4 Categories

A category belongs to one event. The Organiser should choose an event, see its categories, add new categories, and edit/delete categories if those API endpoints exist in the approved plan.

```
var response = await _api.SendAsync(
    HttpMethod.Get, $"api/events/{eventId}/categories");

var categories = await _api.ReadAsync<List<CategoryDto>>(response) ?? new();
```

6.5 Enrolments

For each Organiser event, show the Participants enrolled, the category they selected and the enrolment status. Statuses should be visually distinct in the UI.

```
@if (item.Status == "Confirmed")
{
   <span class="badge bg-success">Confirmed</span>
}
else if (item.Status == "Pending")
{
   <span class="badge bg-warning text-dark">Pending</span>
}
else
{
   <span class="badge bg-secondary">@item.Status</span>
}
```

6.6 Results entry

The Organiser must capture finish time and finishing position for an enrolled Participant after the event. Keep the UI tied to an enrolment so the result cannot accidentally be created for someone who never entered the event.

```
public class CaptureResultVm
{
   public int EnrolmentId { get; set; }
   public TimeSpan FinishTime { get; set; }
   public int FinishingPosition { get; set; }
}
```

7. Build the Participant interface

7.1 Upcoming events page with filtering

The Participant home page should display upcoming events and offer filtering. A simple beginner implementation can load events from the API and filter by event type, location or text query in the MVC controller.

```
public async Task<IActionResult> Index(string? type, string? search)
{
    if (!IsParticipant()) return RedirectToAction("Login", "Auth");

    var response = await _api.SendAsync(HttpMethod.Get, "api/events");
    var events = await _api.ReadAsync<List<EventDto>>(response) ?? new();

    events = events.Where(e => e.EventDate >= DateTime.Today).ToList();

    if (!string.IsNullOrWhiteSpace(type))
        events = events.Where(e => e.EventType == type).ToList();

    if (!string.IsNullOrWhiteSpace(search))
        events = events.Where(e =>
            e.Name.Contains(search, StringComparison.OrdinalIgnoreCase) ||
            e.Location.Contains(search, StringComparison.OrdinalIgnoreCase)).ToList();

    return View(events);
}
```

7.2 Event details and category selection

The details page should show the full event information and available categories. A logged-in Participant chooses a category and presses Enter Event. The MVC controller sends the CreateEnrolmentRequest to the Part 2 API.

```
[HttpPost]
public async Task<IActionResult> EnterEvent(int eventId, int categoryId)
{
    if (!IsParticipant()) return RedirectToAction("Login", "Auth");

    var request = new CreateEnrolmentRequest
    {
        EventId = eventId,
        CategoryId = categoryId
    };

    var response = await _api.SendAsync(HttpMethod.Post, "api/enrolments", request);

    if (!response.IsSuccessStatusCode)
    {
        TempData["Error"] = await response.Content.ReadAsStringAsync();
        return RedirectToAction("Details", "Events", new { id = eventId });
    }

    TempData["Success"] = "You have entered the event successfully.";
```

```
    return RedirectToAction(nameof(MyEvents));
}
```

7.3 My Events

Use the Part 2 participant enrolment endpoint so the page displays the user's own enrolments and current status. Do not add a DbContext to MVC just to fetch the data.

7.4 My Results

The official requirement expects event name, date, category, finish time and finishing position. If your ResultDto does not currently return enough display information, extend the API response DTO rather than querying the database from MVC.

7.5 Profile

The profile page calls GET/PUT profile endpoints from Part 2. Part 3 additionally expects the Participant to be able to upload a profile picture. The upload must go from MVC to API and then from API to Azure Blob Storage.

8. Azure Blob Storage - step by step through the API

Architecture rule

RaceDay.Mvc must NOT connect to Azure Storage directly. MVC sends the file to RaceDay.Api. The API uploads it to Blob Storage, receives/constructs the blob URL, stores that URL in the database, and returns data to MVC.

8.1 Add fields to the stored models if they do not exist

```
// Event.cs
public string? BannerImageUrl { get; set; }

// AppUser.cs
public string? ProfileImageUrl { get; set; }
```

After changing EF Core models, create and apply a migration in RaceDay.Api. Use a meaningful name such as AddImageUrls.

```
Add-Migration AddImageUrls
Update-Database
```

8.2 Install the Azure package in RaceDay.Api

```
Azure.Storage.Blobs
```

8.3 Add Blob settings to RaceDay.Api/appsettings.json

```
"AzureBlobStorage": {
  "ConnectionString": "YOUR_AZURE_STORAGE_CONNECTION_STRING",
  "ContainerName": "raceday-images"
}
```

Do not publish secrets

Do not commit a real production Azure connection string to a public GitHub repository. For assessment/demo environments, use user secrets, environment variables or an institution-approved secret method. The code reads configuration; the secret value itself should not be hard-coded in a controller.

8.4 BlobStorageService reference implementation

```
using Azure.Storage.Blobs;
using Azure.Storage.Blobs.Models;

public class BlobStorageService
{
    private readonly BlobContainerClient _container;

    public BlobStorageService(IConfiguration configuration)
    {
        string connectionString = configuration["AzureBlobStorage:ConnectionString"]
            ?? throw new InvalidOperationException("Blob connection string missing.");

        string containerName = configuration["AzureBlobStorage:ContainerName"]
            ?? "raceday-images";
```

```
        _container = new BlobContainerClient(connectionString, containerName);
    }

    public async Task<string> UploadAsync(IFormFile file, string folder)
    {
        if (file == null || file.Length == 0)
            throw new ArgumentException("No file was supplied.");

        string extension = Path.GetExtension(file.FileName);
        string blobName = $"{folder}/{Guid.NewGuid()}{extension}";

        await _container.CreateIfNotExistsAsync();

        BlobClient blob = _container.GetBlobClient(blobName);

        await using Stream stream = file.OpenReadStream();
        await blob.UploadAsync(stream, new BlobHttpHeaders
        {
            ContentType = file.ContentType
        });

        return blob.Uri.ToString();
    }
}
```

8.5 Register the service in RaceDay.Api/Program.cs

```
builder.Services.AddScoped<BlobStorageService>();
```

8.6 API upload endpoint pattern

```
[HttpPost("{id:int}/banner")]
public async Task<IActionResult> UploadBanner(
    int id, IFormFile file, [FromServices] BlobStorageService blobStorage)
{
    // Keep your existing Organiser/session ownership checks here.
    var eventItem = await _db.Events.FindAsync(id);
    if (eventItem == null) return NotFound();

    string url = await blobStorage.UploadAsync(file, "event-banners");
    eventItem.BannerImageUrl = url;
    await _db.SaveChangesAsync();

    return Ok(new { imageUrl = url });
}
```

Do not remove role checks

The upload endpoint is still part of the protected API. Keep the same Organiser ownership/session rules you used for event editing. A Participant should not be able to upload an Event banner.

8.7 Send a file from MVC to API

```
public async Task<HttpResponseMessage> UploadFileAsync(
    string url, IFormFile file)
{
```

```
    var client = _factory.CreateClient("RaceDayApi");
    using var request = CreateRequest(HttpMethod.Post, url);
    using var form = new MultipartFormDataContent();
    using var stream = file.OpenReadStream();
    var fileContent = new StreamContent(stream);
    fileContent.Headers.ContentType =
        new System.Net.Http.Headers.MediaTypeHeaderValue(file.ContentType);

    form.Add(fileContent, "file", file.FileName);
    request.Content = form;
    return await client.SendAsync(request);
}
```

8.8 MVC event form must use multipart encoding

```
<form asp-action="EditEvent" method="post" enctype="multipart/form-data">
    ... normal event inputs ...

    <label>Event banner</label>
    <input type="file" name="banner" class="form-control" accept="image/*" />

    <button type="submit" class="btn btn-primary">Save</button>
</form>
```

8.9 Display the stored image

```
@if (!string.IsNullOrWhiteSpace(Model.BannerImageUrl))
{
    <img src="@Model.BannerImageUrl" alt="@Model.Name event banner"
         class="img-fluid rounded" />
}
```

Azure access note

A raw blob URL is only directly viewable if the container/blob access configuration allows it. If your Azure Storage policy blocks public access, use an institution-approved approach such as an API download endpoint or signed access. Do not weaken a production storage account just to make a demo easier.

9. Docker beginner bootcamp - start from zero

Read this even if Docker is already installed

A student who can click Run in Docker Desktop but cannot explain image, container, Dockerfile, port mapping or logs will struggle to troubleshoot Part 3. This section builds the mental model first.

9.1 What Docker is

Docker packages an application and the runtime environment it needs into an image. A running instance of that image is a container. Docker does not rewrite your C# code; it changes how the finished application is packaged and run.

Term	Beginner meaning	RaceDay comparison
Dockerfile	Recipe/instructions for building the image	Instructions describing how to build RaceDay.Mvc
Image	Packaged template	The saved RaceDay MVC package ready to run
Container	Running instance of an image	The actual running RaceDay MVC website
Port mapping	Connect host port to container port	localhost:8080 -> container:8080
Docker Desktop	Windows application that runs/manages Docker	Engine + GUI for images/containers/logs

9.2 The complete flow

```
SOURCE CODE
  |
  v
DOCKERFILE
  | docker build
  v
IMAGE
  | docker run
  v
CONTAINER
  | port mapping
  v
BROWSER OPENS RACEDAY
```

9.3 Windows prerequisites

- Windows 10/11 supported by current Docker Desktop.
- Hardware virtualisation enabled.
- WSL 2 installed and current.
- Docker Desktop installed and running.
- Visual Studio 2022 with ASP.NET and web development workload.
- At least 8 GB RAM is recommended by current Docker Desktop requirements.

9.4 Check WSL before installing Docker

```
wsl --version
```

If version details do not appear or WSL is missing, open PowerShell/Command Prompt as Administrator and use:

```
wsl --install
```

If WSL exists but needs updating:

```
wsl --update
```

Restart Windows if requested. Do not keep troubleshooting Docker before completing a required restart.

9.5 Check virtualisation

11. Press Ctrl + Shift + Esc to open Task Manager.
12. Choose Performance.
13. Choose CPU.
14. Find Virtualization. It should show Enabled.
15. If it is Disabled on a personal computer, enable virtualisation in BIOS/UEFI. On a managed college computer, contact technical support rather than changing BIOS settings without permission.

9.6 Install Docker Desktop

16. Download Docker Desktop for Windows from the official Docker website.
17. Run Docker Desktop Installer.
18. For most students, use the recommended per-user installation.
19. Use the WSL 2 backend when offered.
20. Finish installation and launch Docker Desktop.
21. Wait until Docker Desktop reports that the engine is running.

9.7 Verify the installation from a terminal

```
docker --version
docker info
```

If docker --version works but docker info says it cannot connect to the engine, Docker Desktop is usually not fully started yet.

9.8 Run hello-world

```
docker run hello-world
```

Docker will look for the image locally, download it if needed, create a container, run it, print the result, then the short-lived container exits.

9.9 See images and containers

```
docker images
docker ps
docker ps -a
```

Command	What it shows
docker images	Images saved locally
docker ps	Containers that are running now
docker ps -a	All containers including stopped ones

9.10 Run a long-lived web container

```
docker run -d -p 8080:80 --name mywebsite nginx
```

Open http://localhost:8080 in a browser. If you see the Nginx page, you have proved that Docker can publish a container port to your Windows computer.

Understand every part of the command

Part	Meaning
docker run	Create and start a container
-d	Detached: leave it running in the background
-p 8080:80	Host port 8080 maps to container port 80
--name mywebsite	Give the container a memorable name
nginx	Image to run

9.11 Stop, start, inspect and remove

```
docker stop mywebsite
docker start mywebsite
docker logs mywebsite
docker inspect mywebsite
docker rm -f mywebsite
```

Do not be afraid of the terminal

For this assessment, the terminal is not replacing Docker Desktop. It gives you precise evidence and error messages. Docker Desktop and the CLI control the same Docker engine.

10. Dockerise RaceDay.Mvc - the full beginner workflow

10.1 First rule: test the MVC app normally

22. Start RaceDay.Api.
23. Start RaceDay.Mvc normally from Visual Studio.
24. Register/login through MVC.
25. Confirm role navigation.
26. Open at least one Organiser or Participant protected page.
27. Confirm at least one API data operation succeeds.
28. Only continue to Docker after the local website works.

Why?
Docker cannot fix a broken controller, incorrect DTO, bad API route or failing session. If the application already fails locally, containerising it adds another layer of errors and makes debugging harder.

10.2 Choose a simple local HTTP port for the API

For the Docker demonstration, the MVC container needs to call the API running on the Windows host. HTTPS development certificates inside this host/container scenario often confuse beginners, so a simple local assessment setup is to run the API over HTTP on a known port.

```
dotnet run --project RaceDay.Api --urls http://0.0.0.0:5000
```

Leave that terminal running. Test the API from the Windows browser using the route that exists in your project, for example http://localhost:5000/swagger if Swagger is available in that environment.

10.3 Why localhost changes inside a container

When the MVC application runs normally on Windows, http://localhost:5000 can refer to the API on the same Windows computer. When MVC runs inside Docker, localhost refers to the MVC container itself. The API is not inside that container.

```
Windows host:
  RaceDay.Api -> http://localhost:5000

Docker container:
  localhost -> the MVC container itself
  host.docker.internal -> the Windows host
```

Correct Docker API URL
Inside the MVC container use http://host.docker.internal:5000/ to reach the API running on Windows through Docker Desktop.

10.4 Add Docker support from Visual Studio (recommended for beginners)

29. Right-click the RaceDay.Mvc project, not the solution.
30. Choose Add > Docker Support or Add > Container Support (wording depends on your Visual Studio version).
31. Choose Linux containers unless your lecturer specifically instructed Windows containers.

32. Visual Studio creates a Dockerfile.
33. Open the Dockerfile and read it instead of treating it as magic.

If Visual Studio does not show Docker/Container Support

Open Visual Studio Installer > Modify > confirm the ASP.NET and web development workload and container development tools are installed. Also confirm Docker Desktop is installed and running.

10.5 A clean multi-stage Dockerfile for this RaceDay solution

Because RaceDay.Mvc references RaceDay.Contracts, the Docker build must copy both project files before restore. This reference Dockerfile assumes the Docker build context is the solution root.

```
# Runtime image: smaller image used when the finished website runs.
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
USER $APP_UID
WORKDIR /app
EXPOSE 8080

# SDK image: contains compilers/tools required to restore and build the project.
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
ARG BUILD_CONFIGURATION=Release
WORKDIR /src

# Copy project files first so Docker can cache package restore layers.
COPY ["RaceDay.Mvc/RaceDay.Mvc.csproj", "RaceDay.Mvc/"]
COPY ["RaceDay.Contracts/RaceDay.Contracts.csproj", "RaceDay.Contracts/"]

RUN dotnet restore "RaceDay.Mvc/RaceDay.Mvc.csproj"

# Copy the rest of the repository after restore.
COPY . .
WORKDIR "/src/RaceDay.Mvc"
RUN dotnet build "RaceDay.Mvc.csproj" -c $BUILD_CONFIGURATION -o /app/build

# Publish the compiled MVC application.
FROM build AS publish
ARG BUILD_CONFIGURATION=Release
RUN dotnet publish "RaceDay.Mvc.csproj" -c $BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false

# Final runtime container.
FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "RaceDay.Mvc.dll"]
```

10.6 What every Dockerfile line means

Instruction	Meaning
FROM ... aspnet:8.0	Start from Microsoft runtime image used to run ASP.NET Core
USER $APP_UID	Run as the non-root application user supported by modern .NET container images
WORKDIR /app	Set the working folder inside the container
EXPOSE 8080	Documents the port the app listens on
FROM ... sdk:8.0	Use the larger SDK image for restore/build/publish

COPY csproj...	Copy project definitions before the rest of the files
RUN dotnet restore	Download NuGet packages
COPY . .	Copy remaining source files
RUN dotnet build	Compile the MVC project
RUN dotnet publish	Create deployable output
COPY --from=publish	Copy only published output into final runtime stage
ENTRYPOINT	Command that starts RaceDay.Mvc

10.7 Add .dockerignore at the solution root

```
**/bin/
**/obj/
.git/
.vs/
*.user
*.suo
Dockerfile*
README.md
```

A .dockerignore prevents unnecessary build output, IDE files and repository metadata from being sent to the Docker build context. Do not ignore something the build actually needs.

10.8 Open the terminal in the correct folder

For the Dockerfile above, run the build from the folder containing RaceDay.sln and the RaceDay.Mvc and RaceDay.Contracts folders.

```
dir
```

You should be able to see something similar to:

```
RaceDay.sln
RaceDay.Api
RaceDay.Contracts
RaceDay.Mvc
RaceDay.Api.Tests
```

10.9 Build the image

```
docker build -f RaceDay.Mvc/Dockerfile -t raceday-mvc:part3 .
```

Read the command from right to left

Part	Meaning
.	Use the current solution folder as the build context
-t raceday-mvc:part3	Name/tag the image
-f RaceDay.Mvc/Dockerfile	Use this Dockerfile
docker build	Build an image

The final dot is not decoration. If you remove it, Docker has no build context.

10.10 Confirm the image exists

```
docker images
```

Look for raceday-mvc with tag part3.

10.11 Run the MVC container

The MVC container must receive a different API URL from the local appsettings value. ASP.NET Core configuration supports environment-variable overrides using double underscores for nested keys.

```
docker run -d ^
 --name raceday-mvc ^
 -p 8080:8080 ^
 -e ApiSettings__BaseUrl=http://host.docker.internal:5000/ ^
 raceday-mvc:part3
```

The ^ line-continuation syntax above is for Windows Command Prompt. A one-line version is easiest to paste anywhere:

```
docker run -d --name raceday-mvc -p 8080:8080 -e ApiSettings__BaseUrl=http://host.docker.internal:5000/ raceday-mvc:part3
```

Understand the run command

Part	Meaning
-d	Run in background
--name raceday-mvc	Name the container
-p 8080:8080	Windows localhost:8080 maps to MVC container port 8080
-e ApiSettings__BaseUrl=...	Override MVC API base URL inside container
raceday-mvc:part3	Image to start

10.12 Confirm the container is running

```
docker ps
```

You should see the container name and a port mapping similar to 0.0.0.0:8080->8080/tcp.

10.13 Open RaceDay

```
http://localhost:8080
```

Test the same functionality you tested locally: login, role navigation, data display and at least one data-changing operation. The official requirement says the containerised MVC application must behave like the local application.

10.14 Prove the container is doing the work

Stop the normal RaceDay.Mvc Visual Studio debugging session but leave the Docker container running. Refresh http://localhost:8080. If the site still runs, you have strong evidence that Docker is serving the MVC application.

10.15 View logs

```
docker logs raceday-mvc
docker logs -f raceday-mvc
```

Press Ctrl + C to stop following logs. This does not stop the container.

10.16 Stop and start without rebuilding

```
docker stop raceday-mvc
docker start raceday-mvc
```

10.17 What happens after you change C# code?

An existing image contains the published code that existed when it was built. Changing a controller or Razor view on Windows does not magically update the old image.

```
docker stop raceday-mvc
docker rm raceday-mvc
docker build -f RaceDay.Mvc/Dockerfile -t raceday-mvc:part3 .
docker run -d --name raceday-mvc -p 8080:8080 -e ApiSettings__BaseUrl=http://host.docker.internal:5000/ raceday-mvc:part3
```

README requirement

Put the final verified docker run command in README.md exactly as you tested it. Do not submit a guessed command.

11. Docker troubleshooting - diagnose instead of guessing

Problem	Likely cause	What to do
docker is not recognised	Docker CLI missing or terminal opened before install	Start Docker Desktop, reopen terminal, run docker --version
Cannot connect to Docker engine	Docker Desktop not running/ready	Open Docker Desktop and wait; run docker info
WSL error	WSL missing/outdated	Run wsl --version then wsl --update; restart if required
Virtualisation disabled	BIOS/UEFI or managed PC setting	Check Task Manager; contact IT on managed machines
Port already allocated	Another app/container uses 8080	docker ps; stop old container or use -p 8081:8080
Container name already in use	Old stopped container still exists	docker ps -a then docker rm -f raceday-mvc
Container immediately exits	MVC crashed during startup	docker ps -a then docker logs raceday-mvc
Dockerfile cannot find csproj	Wrong build folder/context or COPY path	Run dir; build from solution root; check exact project names
MVC opens but API calls fail	Container tries localhost or API not exposed to host network	Use host.docker.internal and start API on 0.0.0.0:5000
401 after login	API session cookie not being preserved	Check Set-Cookie capture and Cookie header forwarding in RaceDayApiClient
HTTPS certificate error	Container cannot trust/use dev cert as expected	Use the documented local HTTP API demo port for assessment testing
Images do not display	Blob is private or URL/property not persisted	Verify Azure upload, database URL and storage access policy
Changes not visible	Old Docker image/container still running	Stop/remove, rebuild image, start new container

11.1 The four commands to use first

```
docker ps
docker ps -a
docker logs raceday-mvc
docker inspect raceday-mvc
```

11.2 If the browser says connection refused

34. Run docker ps. Is raceday-mvc actually running?
35. Check PORTS. Do you see 8080->8080?
36. Run docker logs raceday-mvc. Did ASP.NET Core start?
37. If another service uses port 8080, rerun with -p 8081:8080 and browse to localhost:8081.

11.3 If MVC cannot reach the API

38. On Windows, confirm the API works at http://localhost:5000.
39. Confirm the API was started with http://0.0.0.0:5000 for the Docker test.
40. Inspect the container environment and confirm ApiSettings__BaseUrl uses host.docker.internal, not localhost.
41. Read MVC logs. A connection refused error normally means address/port/listening configuration, not a Razor styling issue.

11.4 Docker Desktop inside a virtual lab

Docker Desktop may fail in a virtual machine that does not provide nested virtualisation. If the college lab or cloud VM does not support nested virtualisation, use an approved physical machine or supported environment rather than repeatedly reinstalling Docker.

12. UI design: make RaceDay look like one professional system

The top UI band is not about adding random colours. It is about consistent RaceDay branding, readable layouts, clear navigation, role-aware screens, responsive Bootstrap components and visual status cues.

12.1 Suggested RaceDay visual direction

- Sporting/energetic rather than corporate generic.
- Use one strong primary colour and one accent consistently.
- Keep headings, button styles and card spacing consistent.
- Use Bootstrap grid/cards/tables/forms.
- Use status badges: Confirmed green, Pending amber, Cancelled/Rejected red/grey.
- Use event banner images meaningfully on event cards/details.
- Make Organiser and Participant navigation clearly different.

12.2 Reusable status CSS

```
.status-confirmed { background:#198754; color:white; }
.status-pending   { background:#ffc107; color:#222; }
.status-cancelled { background:#6c757d; color:white; }
```

12.3 Error handling

Do not show raw stack traces or database errors to users. Controllers should turn failed API responses into ModelState errors, TempData messages or friendly error pages. Logs are for technical diagnosis; user pages should remain understandable.

13. GitHub and CI/CD for Part 3

The official Part 3 requirement is at least 20 meaningful commits and a GitHub Actions workflow updated to build the MVC project. Keep Part 1 and Part 2 history; do not delete your earlier evidence.

13.1 Meaningful commit examples

- Create RaceDay MVC project
- Configure MVC API client and session
- Add MVC registration and login
- Add role-aware navigation
- Build organiser dashboard
- Add event management views
- Add category management
- Add organiser enrolment view
- Add results entry
- Add participant event browsing
- Add participant enrolment flow
- Add My Events
- Add My Results
- Add participant profile
- Add event banner API upload
- Add profile image upload
- Add Azure Blob service
- Add RaceDay UI styling
- Add Dockerfile for MVC
- Verify MVC Docker container
- Update Part 3 CI workflow
- Complete Part 3 README and video evidence

13.2 GitHub Actions reference workflow

```
name: RaceDay CI

on:
  push:
  pull_request:

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '8.0.x'
```

```
- name: Restore
  run: dotnet restore RaceDay.sln

- name: Build
  run: dotnet build RaceDay.sln --configuration Release --no-restore

- name: Test
  run: dotnet test RaceDay.Api.Tests/RaceDay.Api.Tests.csproj --configuration Release --no-build
```

Docker build in CI is optional unless specifically required

The official Part 3 brief says update CI/CD to build the MVC project. A successful solution build already validates RaceDay.Mvc if it is included in RaceDay.sln. You can add a Docker build job for stronger evidence, but do not let an unnecessary CI complication break an otherwise correct required workflow.

14. README and video presentation

14.1 README structure

```
# RaceDay

## Project Overview
## User Roles
### Organiser
### Participant

## Solution Structure
## Part 1 - Planning and Database
## Part 2 - RESTful API
## Part 3 - MVC Web Application

## Prerequisites
## Database Setup
## How to Run RaceDay.Api
## How to Run RaceDay.Mvc Locally
## Azure Blob Storage Setup
## Docker Setup
### Build Image
### Run Container
## Authentication and Session Flow
## Organiser Features
## Participant Features
## CI/CD
## GitHub Actions Screenshot
## Video Presentation
```

14.2 Put a verified Docker command in README

```
docker run -d --name raceday-mvc -p 8080:8080 -e ApiSettings__BaseUrl=http://host.docker.internal:5000/ raceday-mvc:part3
```

Also explain that the API must be reachable at the configured base URL. A command without this context may run the MVC container but leave all data pages broken.

14.3 Video order that demonstrates the rubric efficiently

42. Introduce RaceDay and explain Part 3 architecture: MVC -> API -> SQL/Azure.
43. Show the solution structure including RaceDay.Mvc.
44. Demonstrate registration and login.
45. Show Organiser navigation and dashboard.
46. Create/edit an event and demonstrate the banner image.
47. Show categories, enrolments and result capture.
48. Log out and log in as Participant.
49. Browse/filter events and open event details.
50. Enter an event and show My Events.
51. Show My Results and Profile/Profile Picture.
52. Show the Azure Blob integration code in the API and explain why MVC does not connect directly to Azure.
53. Stop local MVC and demonstrate Docker Desktop/terminal.

54. Show docker images and docker ps.
55. Open RaceDay through the container port.
56. Demonstrate at least one API-backed page while MVC is containerised.
57. Show Dockerfile and explain base/build/publish/final stages.
58. Show GitHub commit history and green CI build.

15. Final top-band checklist

Correct submission - 5

- GitHub link opens.
- RaceDay.sln contains API, Contracts, Tests and MVC.
- README is complete.
- Docker run command is present and tested.
- CI screenshot is in README.
- Video link is present.

GitHub and CI/CD - 10

- 20+ meaningful Part 3 commits.
- Workflow restores and builds the whole solution.
- RaceDay.Mvc is part of the solution and therefore built.
- API tests still pass.
- Latest workflow is green.

Authentication and Role Navigation - 10

- Register works from MVC.
- Login works through API.
- MVC stores UserId and Role in session.
- API session cookie is preserved for protected API requests.
- Organiser and Participant see different menus.
- Protected pages redirect unauthenticated users.
- Wrong role is blocked.
- Logout clears session and redirects appropriately.

Organiser Interface - 15

- Dashboard works.
- Events create/edit/delete work.
- Required event fields are present.
- Banner image works.
- Category management works.
- Enrolment list works.
- Results entry works.

Participant Interface - 15

- Upcoming events load.
- Filters work.
- Event details show categories.
- Enter Event works.
- My Events works.
- My Results shows event/date/category/time/position.
- Profile update works.

- Profile picture upload works.

Azure Blob Storage - 10

- File travels MVC -> API -> Azure.
- MVC does not use BlobServiceClient directly.
- API uploads file.
- URL/path is saved with correct database record.
- Image displays in MVC.
- Secrets are not committed publicly.

Docker - 10

- Docker Desktop runs.
- docker --version and docker info work.
- Dockerfile exists.
- Image builds without errors.
- Container stays running.
- Port mapping works.
- MVC reaches API using host.docker.internal in the local Docker demonstration.
- Login and at least one protected/data page work inside container.
- README run command has been tested.

UI Design - 10

- Consistent sporting RaceDay theme.
- Responsive Bootstrap layout.
- Role-specific navigation.
- Status colours are clear.
- Event images improve presentation.
- No raw exception messages shown to user.

Video - 15

- Shows application running, not only code.
- Shows both roles.
- Shows Blob upload and result.
- Shows Docker build/run/container.
- Explains important code and architecture.
- Shows CI and repository evidence.

Final rule

Do not submit when only the local version works. The Docker marks depend on the containerised MVC application actually working, and the overall Part 3 marks depend on the MVC application still communicating correctly with the Part 2 API.

Appendix A. Known-good Docker command sequence

Use this sequence when you need to rebuild from a clean container after changing code.

```
# 1. Confirm Docker Desktop is running
docker info

# 2. From the solution root, build the MVC image
docker build -f RaceDay.Mvc/Dockerfile -t raceday-mvc:part3 .

# 3. Remove any old container with the same name
docker rm -f raceday-mvc

# 4. Start the Part 2 API on the Windows host
dotnet run --project RaceDay.Api --urls http://0.0.0.0:5000

# 5. In a second terminal, run the MVC container
docker run -d --name raceday-mvc -p 8080:8080 -e ApiSettings__BaseUrl=http://host.docker.internal:5000/ raceday-mvc:part3

# 6. Confirm it is running
docker ps

# 7. Read startup logs
docker logs raceday-mvc

# 8. Open in browser
http://localhost:8080
```

Appendix B. Docker evidence students should capture

- Docker Desktop open with engine running.
- Terminal showing docker --version.
- Successful docker build ending without error.
- docker images showing raceday-mvc:part3.
- docker ps showing raceday-mvc running and the port mapping.
- Browser showing RaceDay from localhost container port.
- A logged-in Organiser or Participant page working while MVC is containerised.
- docker logs output showing the MVC app started successfully.
- README section containing the verified Docker build/run instructions.

Appendix C. Sources used for this student guide

Assessment basis: The Independent Institute of Education, PROG6212 Programming 2B PoE 2026, Part 3 / Final PoE requirements and marking rubric; existing PROG6212 Part 2 Detailed Student Guide used for continuity of project structure and session/API conventions.

Technical references: Docker Desktop for Windows documentation; Docker Desktop WSL 2 documentation; Docker Desktop networking documentation for host.docker.internal; Microsoft Learn ASP.NET Core container/Visual Studio Docker guidance; Microsoft Learn ASP.NET Core session state and IHttpClientFactory guidance; Microsoft Learn Azure Blob Storage .NET upload guidance.

This guide uses a reference implementation. Students must keep route names, DTO properties, database entities and business rules consistent with the Part 1 plan and Part 2 API they actually submitted.